

MOOC Assistant — Third Semester

Presentation Script · Recorded Version

Tarun VENKATESH · IMT Atlantique · 2025–2026

Total target speaking time: **10–12 minutes** + live demo

How to use this script: Read it naturally — do not rush. Pause at every paragraph break. Words in **bold** carry emphasis. **[PRESENTER NOTES]** are reminders to yourself, not to be read aloud.

SLIDE 1 — TITLE

Approx. speaking time: ~45 seconds

Hello everyone. My name is **Tarun Venkatesh**, and I am a student at **IMT Atlantique**, working under the supervision of Professor Laurent Toutain and Baptiste Gaultier.

This is my third-semester project presentation for the **MOOC Assistant** — an intelligent, multi-agent system designed to help automate responses to student questions on the FUN-MOOC online learning platform.

Let me walk you through what this project is about, what I built this semester, and where it is headed.

[PRESENTER NOTE: Speak slowly. Let the title slide breathe for a moment before moving on.]

SLIDE 2 — INTRODUCTION

Approx. speaking time: ~1 minute 15 seconds

The challenge this project addresses is a very real one. When a MOOC course attracts hundreds or even thousands of students, the discussion forum fills up with questions — technical questions about the code, conceptual questions about the material, and sometimes administrative questions about the course itself.

Instructors simply cannot respond to all of them manually. Questions go unanswered, students get stuck, and the learning experience suffers.

The MOOC Assistant is my answer to this problem. It is a **pipeline of specialised agents** that works together to — first, automatically extract student forum posts; second, generate responses that are grounded in the actual course material; and third, route anything uncertain or complex to the instructor for human review.

The system is specifically built for a MicroPython and IoT course on the **FUN-MOOC platform**, and it generates responses in French — matching the language of the course.

Importantly, this is **not a chatbot**. It is a pipeline where every agent has a specific responsibility, and automation is applied only where it can be trusted.

SLIDE 3 — PREVIOUS SEMESTER RECAP

Approx. speaking time: ~1 minute

Before diving into this semester's work, let me briefly recap what I built in the first semester, because this semester's work builds directly on top of it.

In Semester 1, I established the **extraction foundation** of the system. Using the **Model Context Protocol** — or MCP — and **Playwright browser automation**, I built a system that could log into the FUN-MOOC platform, navigate through the forum's paginated topic listing

pages, and extract individual forum posts.

Each post was parsed from the page's accessibility tree into a structured record containing the author's username, the date, and the post content. The extractor also handled multi-page topics by detecting pagination links and navigating each page sequentially.

By the end of Semester 1, the system could reliably log in, discover forum topics, and produce structured post data — but there was **no RAG, no validation logic, and no quality control**. That is exactly what this semester focused on.

SLIDE 4 — SEMESTER 3 — IMPLEMENTATIONS

Approx. speaking time: ~1 minute 30 seconds

This semester, I designed and integrated seven new components into the system.

The first is the **Semantic RAG Pipeline** — a course ingestion and retrieval system that lets the solution agent answer questions using actual course material rather than relying on the LLM's general knowledge.

The second is **Forum Post Indexing** — after each run, answered posts and their AI responses are added to the knowledge base, making the system smarter over time.

Third is the **Intent-Aware Post Filtering** layer — before generating any response, each post is classified. Instructor posts are skipped. Greetings, complaints, and administrative questions are routed to professor review. Only genuine technical questions are forwarded for generation.

Fourth, the **RAG-Augmented Solution Agent** — for qualifying posts, it retrieves relevant course context and generates a grounded response using the Groq-hosted Llama 3.1 model.

Fifth, the **Two-Stage Validation Agent** — every draft response goes through a rule-based check and then an LLM-based scoring step before it reaches the dashboard.

Sixth, the **Human-in-the-Loop** routing — flagged posts, along with their draft answers and validation scores, are sent to the Professor Review queue so the instructor has full context to make a decision.

And finally, I **redesigned the Streamlit dashboard** into three clear, mutually exclusive views — Conversations and Responses, Professor Review, and Others — with accurate sidebar counters.

[PRESENTER NOTE: Move through this slide at a steady pace — do not read every word. Use the cards on the slide as visual anchors.]

SLIDE 5 — FINAL SYSTEM ARCHITECTURE

Approx. speaking time: ~2 minutes

Let me now walk through the full system architecture, because understanding the flow is key to understanding every design decision.

At the top sits the **Orchestrator Agent**. It is the brain of the pipeline. It drives everything — it triggers login and session warm-up, directs the extractor, and iterates over every extracted post, routing it through the rest of the pipeline.

The **Extractor Agent** uses Playwright via the MCP browser tool. It handles login, navigates the FUN-MOOC forum — which is embedded inside the LMS as an iframe — scans topic listing pages, discovers topic URLs, and extracts all posts across paginated threads.

Each post goes to the **Solution Agent**. It first checks whether the post even needs a response. If it does, it queries the RAG index to fetch relevant course chunks, builds a structured prompt, and calls the Groq-hosted LLM to generate an answer.

That draft answer goes to the **Validator Agent**. It runs two stages of checks — which I will explain in a moment — and returns a quality score. Based on that score, the orchestrator decides whether to publish the answer automatically or flag it for professor review.

On the right side, you can see the **RAG Index** — a serialised index of course chunks and previously answered forum posts. The solution agent queries it at inference time, and the orchestrator grows it after each topic is processed.

All of this feeds into the **Streamlit Dashboard**, which shows three separate views depending on the outcome of each post.

The key design principle here is **sequential trust** — each stage only passes forward what it is confident about. Everything else is escalated.

SLIDE 6 — RAG IMPLEMENTATION

Approx. speaking time: ~1 minute 45 seconds

Now let me explain why RAG was needed and exactly how I implemented it.

The core problem with a plain language model is that it answers from its training data alone. In an educational context, this is dangerous — the model may **hallucinate technically incorrect explanations**, and it has no knowledge of the specific course material, exercises, or pedagogical choices of this particular MOOC.

My first attempt used **TF-IDF** — a classic keyword-matching approach. But it consistently failed when students rephrased the same concept differently. For example, a student asking 'comment envoyer des données par HTTP' and a course chunk explaining 'les requêtes GET et POST avec urequests' share almost no common tokens — so TF-IDF would return near-zero similarity even though the content is directly relevant. I observed this producing many irrelevant or empty retrievals during early testing.

I switched to a **semantic embedding model** — specifically **paraphrase-multilingual-MiniLM-L12-v2** from SentenceTransformers. This model produces 384-dimensional dense vectors that encode meaning rather than just word frequency. I chose the multilingual variant specifically because the course and forum mix French educational language with English technical vocabulary — function names, library names, protocol names.

For **ingestion**, I wrote a script that recursively scans the edX course archive. For HTML lesson files, it strips all tags and collects visible text. For XML exercise files, it extracts names, labels, choices, hints, and solutions. Each document is split into overlapping chunks of **600 characters with a 150-character overlap** — the overlap ensures important concepts are never cut across two chunks.

All chunks are encoded in batches and stored as a normalised NumPy matrix. At query time, the student post plus the topic title are encoded into a single vector, and **cosine similarity** is computed against the entire index in one matrix multiplication. Chunks scoring above 0.25 are kept. If none cross the threshold, the top-k are returned anyway to prevent empty retrieval.

The final context — up to **4,000 characters** — is injected into the prompt sent to the LLM. The model is explicitly instructed to answer only from what it receives, and to emit a special skip token if the context is insufficient. That token routes the post to professor review.

After each topic is processed, answered posts and their AI responses are also appended to the index — this is the **forum post indexing** step. It means on the next run, the system can leverage previously resolved questions as secondary context. It is a lightweight form of continual learning.

[PRESENTER NOTE: This is the most technical slide. Speak slowly. The TF-IDF vs embeddings comparison is a strong point — emphasise it.]

SLIDE 7 — VALIDATION AGENT

Approx. speaking time: ~1 minute 30 seconds

Even with RAG, a generated response can be fluent and confident while still being **factually wrong or pedagogically misleading**. In an educational setting, publishing an incorrect answer is worse than publishing no answer at all — students may learn the wrong concept, and the instructor then has to spend extra effort correcting the misinformation.

That is why every draft response goes through the Validation Agent before it ever appears on the dashboard.

The validator works in **two stages**.

Stage 1 is **rule-based** — fast, deterministic, no LLM call. It checks for two things: first, if the draft contains the phrase 'Je vais vérifier cela dans les ressources du cours' — that is the solution agent's fallback when it had no real answer — it is immediately flagged as invalid and

replaced with a polite redirection message. Second, if the answer contains first-person confusion phrases like 'je ne sais pas' or 'je n'arrive pas', it is rejected because it reads like a confused student, not a tutor.

Stage 2 is **LLM-based scoring**. The validator sends the student's question, a 1,200-character excerpt of the retrieved course context, and the draft answer to a stronger model —

llama-3.3-70b-versatile on Groq. I used a larger model here deliberately: generation needs to be fast and frequent, but validation needs higher precision over fewer calls.

The LLM returns a structured JSON object with three scores: **answer_relevance** — does it address the actual question; **grounding_score** — is it supported by the course context; and **clarity_score** — is it pedagogically clear. Temperature is set to 0.1 for consistent, deterministic evaluations.

After scoring, the orchestrator applies one more check: if answer relevance is at or below **0.4**, the post is flagged for professor review regardless of other scores. If both relevance and clarity are above 0.6, the answer is published automatically. If the validator rewrote the answer, it is published with a 'Corrected' label.

[PRESENTER NOTE: The logic chain here — rule check first, then LLM — is the key design argument. Make sure you state WHY you did it in that order (cheap checks first, expensive LLM call only when needed).]

SLIDE 8 — PROBLEMS FACED & SOLUTIONS

Approx. speaking time: ~1 minute 15 seconds

Let me now walk through four key problems I encountered and how I resolved them.

The first was **over-aggressive post filtering**. My early keyword filter was discarding valid posts that happened to start with polite language. A post beginning with 'Bonjour, j'ai un problème avec le flux HTTP' was being thrown away as a simple greeting. The solution was to build **intent-aware logic** that explicitly checks for question markers, issue keywords, and request language — a post is only discarded as a greeting if none of these signals are present.

The second was **session authentication**. The FUN-MOOC forum is embedded inside the LMS as an iframe, which means it uses a completely separate authentication cookie. Navigating directly to the forum after login gave HTTP 403 errors because the forum session cookie had not been set. I fixed this with a **two-step warm-up**: first navigate to the LMS courseware root, then to the specific forum section page. Only then does the extractor proceed to standalone forum URLs.

The third was **dashboard count mismatches**. Sidebar counters were showing totals that didn't match what was displayed — because some posts were being counted in multiple buckets simultaneously. I enforced **mutually exclusive category assignment** with a strict priority order: instructor posts are always ignored first; admin and low-value posts go to

professor review; only posts that pass all filters and validation become auto-responses.

And fourth, the **validator LLM occasionally returned malformed JSON** — wrapped in markdown code fences. I added a post-processing step using a regular expression to strip the fences before parsing, and applied default values to all expected fields so missing keys never crash the pipeline.

SLIDE 9 — FUTURE ENHANCEMENTS

Approx. speaking time: ~45 seconds

Looking ahead, there are three enhancements I would prioritise.

First, **live post monitoring**. Right now, the system is batch-based — it runs manually. The natural next step is to run it on a schedule — every 30 to 60 minutes — comparing newly discovered posts against a record of already-processed ones, and feeding any new posts through the pipeline automatically.

Second, an **instructor feedback loop**. When an instructor reviews and edits a draft answer in the Professor Review queue, that corrected answer could be added to the RAG index as a high-quality, verified example. Over time, this would enrich the knowledge base with teacher-level explanations and progressively improve automated response quality.

Third, **video transcript indexing**. The biggest gap in the current knowledge base is that course videos are not indexed — most course knowledge is delivered through video lectures. If transcripts were available, indexing them would dramatically expand what the system can answer.

SLIDE 10 — CONCLUSION

Approx. speaking time: ~30 seconds

To conclude — this semester, the MOOC Assistant went from a basic extraction prototype to a **complete, production-ready multi-agent pipeline**.

The system now covers the full journey: browser-based forum extraction, RAG-augmented response generation, two-stage validation, and a structured dashboard with human-in-the-loop review.

The design philosophy is deliberate: **automate only where you can trust the automation**. Everything uncertain goes to the instructor — not with an empty queue, but with a draft answer, validation scores, and full context to make an informed decision.

That balance between automation and oversight is, I believe, the right approach for AI in an academic environment. Thank you.

[PRESENTER NOTE: Pause briefly after 'Thank you' before starting the demo.]

LIVE DEMO — DASHBOARD WALKTHROUGH

Great — now let me show you the system running on real forum data. I'll walk you through the three views of the dashboard.

VIEW 1 — CONVERSATIONS AND RESPONSES

This is the main view. After running an extraction, the system processed 9 posts across 3 topics. You can see 6 posts have received automated responses — displayed here with their validation scores.

You will notice two types of response badges: **VALIDATED** and **CORRECTED**. Let me explain the difference.

A **VALIDATED** response — like the one you can see with Relevance 0.80 and Clarity 0.90 — means the draft generated by the solution agent passed both the rule-based check and the LLM scoring without any changes. The answer was good enough as-is, and it was published exactly as generated.

A **CORRECTED** response — like the one showing Relevance 0.70 and Clarity 0.80 — means the validator's LLM identified issues with the original draft and rewrote it before publication. The version you see in the dashboard is the improved, corrected version. The original draft had something the validator judged as unclear or imprecise — so it fixed it automatically.

Both **VALIDATED** and **CORRECTED** responses are published automatically — the difference is simply whether the validator intervened to improve the text.

VIEW 2 — PROFESSOR REVIEW

Here you see 2 posts that the system flagged for instructor attention, tagged as **LOW RELEVANCE**.

Look at the first one — a post from 'blabey' describing what they did to fix their own problem. The AI generated a draft response, but when the validator scored it, the relevance came out at 0.40 and grounding at 0.20. That is below the threshold, so it was automatically escalated here.

The key design here is that the professor is not shown an empty queue — they see the student's full post, the AI's draft response, and all three scores. They have everything they need to decide whether to publish the draft as-is, edit it, or write a completely new response.

The second post — 'Cool ... bonne suite de MOOC' — is a casual comment with no actionable question. The AI produced a response, but it scored 0.00 across all metrics because the validator recognised it as context-free. It is correctly routed here rather than auto-published.

VIEW 3 — OTHERS

The Others view shows posts that were ignored entirely — not sent to generation at all. In this run, you can see one post from **bgaultier** — that is the instructor — labelled as INSTRUCTOR.

The system identified that post as coming from a known instructor username and skipped it immediately. The orchestrator never even passed it to the solution agent. This is the intent-aware filtering layer working correctly.

METRICS SIDEBAR

Finally, the sidebar gives a real-time breakdown: 3 topics processed, 9 posts extracted total — 6 automated responses, 2 in professor review, 1 ignored. All numbers are mutually exclusive — every post appears in exactly one category, which was one of the bugs I had to fix during development.

That is the full pipeline running end-to-end. Thank you for watching.

END OF SCRIPT